AI • FINANCIAL CRIME ANALYTICS • DECISION SUPPORT

PaySim Fraud Intelligence Platform

From synthetic transaction history to explainable risk decisions and an interactive monitoring dashboard.

MACHINE LEARNING | SQL SERVER | DASH | PLOTLY

Project presentation • 30 October 2026 • Prepared for academic evaluation

What the platform does

A historical, post-transaction fraud analytics prototype built on PaySim.

DETECT

Score each evaluated transaction and assign LOW / MEDIUM / HIGH risk.

EXPLAIN

Expose raw model margin, risk level and recommended simulated action.

MONITOR

Store results in SQL Server and visualize KPIs, trends, transactions and alerts.

THE IMPORTANT SCOPE STATEMENT

This is a historical PaySim evaluation and post-transaction decision-support prototype. It is not connected to a real bank, does not authorize payments, and does not actually block or reverse funds. “Simulated hold” is a recorded project outcome only.

Why fraud detection is difficult

The target is rare, decisions are asymmetric, and transaction context matters.

The analytical problem

- Fraud is a tiny fraction of all activity: about 0.129% in the full source file.
- Missing a fraudulent transaction can be costly; flagging too many normal transactions creates review workload.
- Amount alone is not enough: transaction type and balance behavior change the interpretation.

The system objective

- Rank transactions using a model trained on historical examples.
- Choose a threshold using validation data, not the final test set.
- Turn model scores into operational risk tiers and searchable evidence.
- Give analysts a transparent view of why a row was prioritized.

Project plan and stakeholders

The delivery is organized as a layered analytics product, with later security labs kept separate.

PROJECT PHASES

1. Data understanding and EDA
2. Feature engineering and temporal split
3. Model training and validation
4. Risk policy and alerts
5. SQL Server persistence
6. Dash dashboard and evidence

PRIMARY STAKEHOLDERS

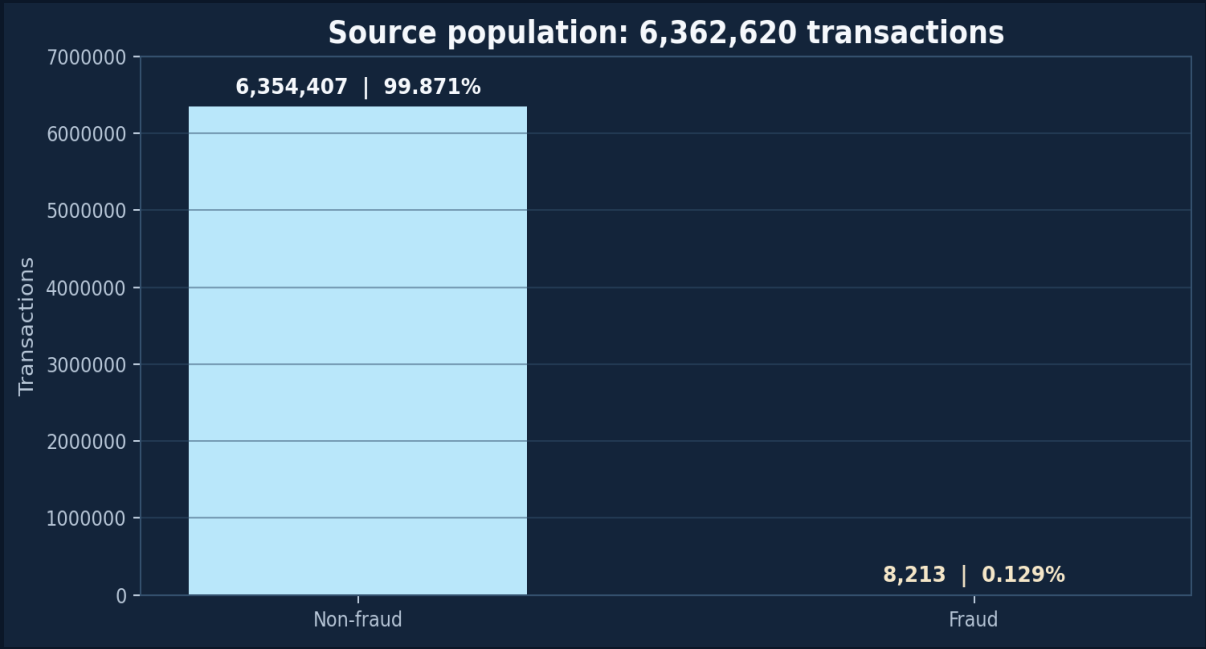
- Project owner / developer
- Evaluators / instructors
- Fraud analyst (intended user)
- Data / ML engineer
- Database administrator
- Security reviewer

SUCCESS CRITERIA

- Reproducible model evaluation
- Leakage-aware split
- Traceable SQL records
- Interpretable risk tiers
- Working interactive dashboard
- Clear limitations and next steps

PaySim dataset at a glance

Source file: 6,362,620 rows, 11 original columns; simulated financial transactions.



LABEL

`isFraud` is the historical target: 1 = fraud, 0 = non-fraud.

TRANSACTION CONTEXT

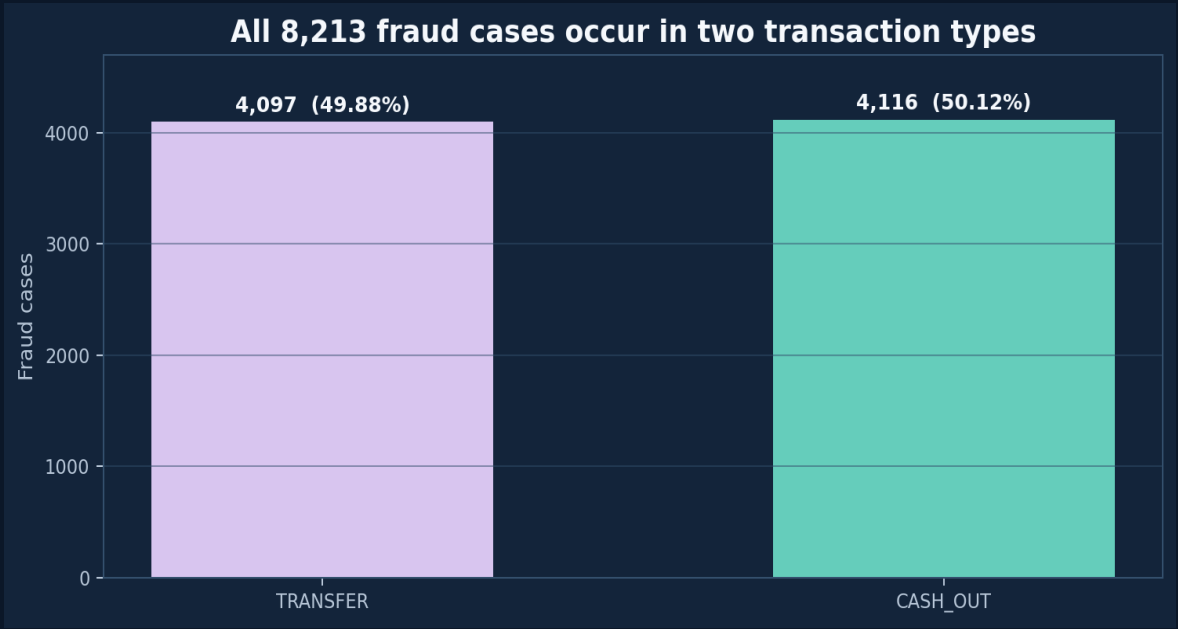
Type, amount, origin/destination IDs, and balances before/after the transaction.

CLASS IMBALANCE

8,213 fraud rows out of 6,362,620 = approximately 0.1291%.

EDA findings: fraud is concentrated

All observed fraud labels in this source file occur in TRANSFER and CASH_OUT.



TRANSFER

4,097 fraud cases. Fraud average amount: 1,480,891.67; normal average: 906,229.01.

CASH_OUT

4,116 fraud cases. Fraud average amount: 1,455,102.59; normal average: 173,917.16.

EDA did not “prove” that large amounts cause fraud. It identified associations worth testing and modeling.

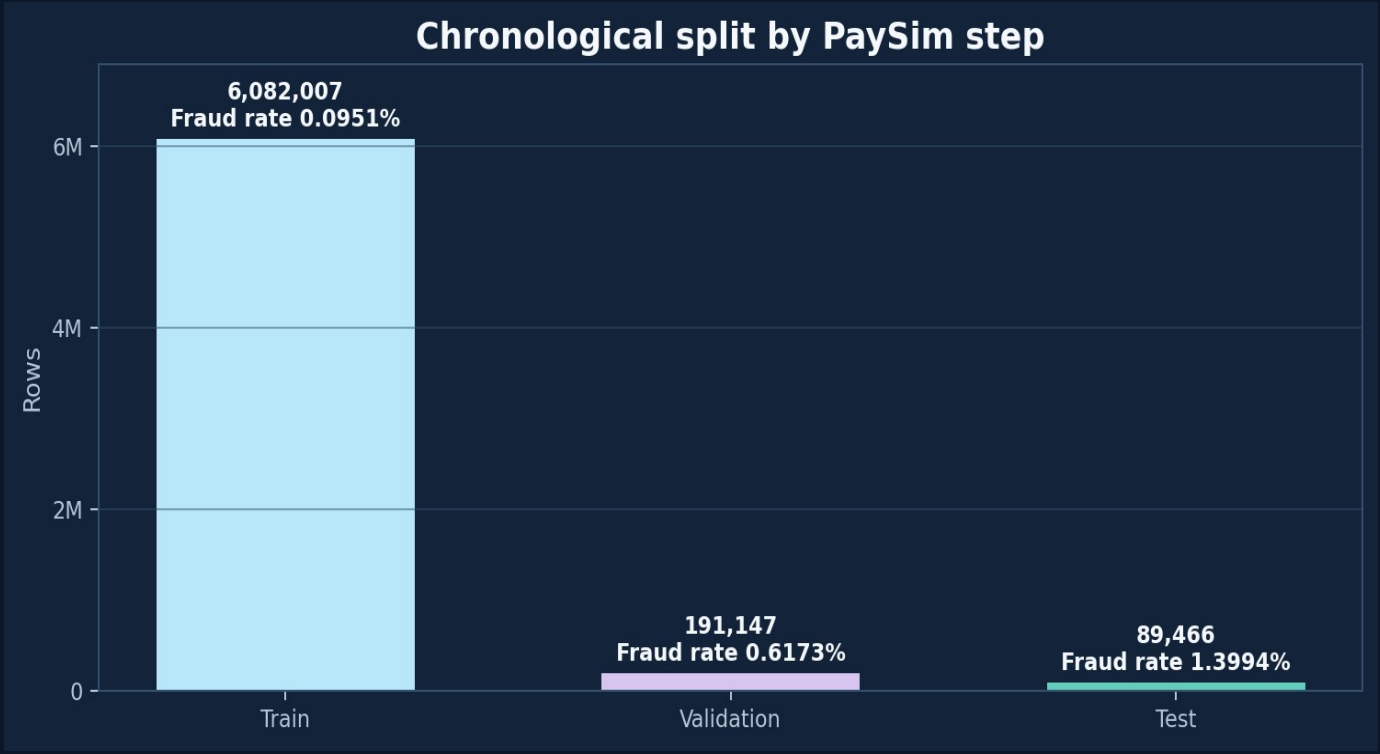
Four hypotheses and what the data actually showed

Separate observed evidence from interpretation; do not over-generalize one feature.

H1 • Amount	Fraud averages were higher in TRANSFER and CASH_OUT.	Supported as an association; amount is not a standalone rule.
H2 • Origin emptied	TRANSFER: 96.02% fraud vs 42.50% normal. CASH_OUT: 99.08% vs 42.78%.	Strong separation in these types; use with context.
H3 • Origin mismatch	Mismatch was far more common in normal rows: TRANSFER 96.21% vs 0.51%; CASH_OUT 89.16% vs 0.58%.	Counter-intuitive; not a universal “fraud anomaly” indicator.
H4 • Destination mismatch	TRANSFER: 99.95% fraud vs 22.07% normal. CASH_OUT: 13.24% vs 22.70%.	Highly transaction-type dependent.

Chronological split: train on the past, test on the future

Rows are split by PaySim step, not randomly, to better reflect temporal deployment.



TRAIN

Steps 1–520 • 6,082,007 rows • 5,781 fraud.

VALIDATION

Steps 521–631 • 191,147 rows • 1,180 fraud.

FINAL TEST

Steps 632–743 • 89,466 rows • 1,252 fraud.

Fraud prevalence rises from 0.0951% → 0.6173% → 1.3994%: a strong temporal distribution-shift warning.

Why this is post-transaction monitoring

Balance-after-transaction fields are only available after the transaction has been processed.

BEFORE

Amount + old balances

TRANSACTION

Transfer / cash-out

AFTER

New balances available

FEATURE AVAILABILITY

New-balance-derived features describe post-transaction evidence. The current product is therefore a post-transaction monitor, not a pre-authorization blocker.

TARGET / ID EXCLUSION

`isFraud` is the target, never a model input. `nameOrig` and `nameDest` are identifiers for traceability, not predictive features. `isFlaggedFraud` was excluded from model features.

DATASET WARNING: PaySim / Kaggle metadata says balance-before/after fields should not be used for fraud detection because fraud transactions are cancelled. Current results therefore need a no-balance-feature ablation.

From raw balances to model-ready signals

The prepared feature table contains 13 columns; identifiers and target stay out of X.

RAW INPUTS

step
type
amount
oldbalanceOrg
newbalanceOrig
oldbalanceDest
newbalanceDest
isFraud (target)

ENGINEERED SIGNALS

log_amount
orig_balance_emptied
orig_balance_error
dest_balance_error
orig_balance_mismatch
dest_balance_mismatch
plus step and type

MODEL INPUT X

Numeric + encoded type

Scaled numerical features

No raw account IDs

No target leakage

Engineering checks: 6,362,620 rows preserved • 8,213 fraud labels preserved • no nulls in prepared feature output.

The classifier: SGDClassifier with logistic loss

A linear decision model trained efficiently through stochastic / mini-batch updates.

WHY THIS MODEL

- Scales to millions of rows
- Supports `partial_fit` batches
- `loss="log_loss"` gives logistic-style classification
- Supports class weighting

WHAT IT LEARNS

A weighted linear score: z = w·x + b

The model learns feature weights w and an intercept b by minimizing logistic loss with regularization.

WHAT IT RETURNS

- `decision_function(X)` → raw margin z
- `predict_proba(X)` → sigmoid-transformed score
- A threshold turns score into a positive / negative decision.

Important: this is not a tree ensemble and not a neural network. It is a linear classifier optimized by stochastic gradient descent.

The mathematics: logistic loss + SGD

Conceptual equations used to understand the model; the implementation delegates optimization to scikit-learn.

1 • LINEAR MARGIN

$z = w^T x + b$

A positive or negative margin indicates which side of the learned boundary the sample falls on.

2 • SIGMOID MAP

$p = 1 / (1 + e^{-z})$

Maps an unbounded margin into a number between 0 and 1.

3 • LOGISTIC LOSS

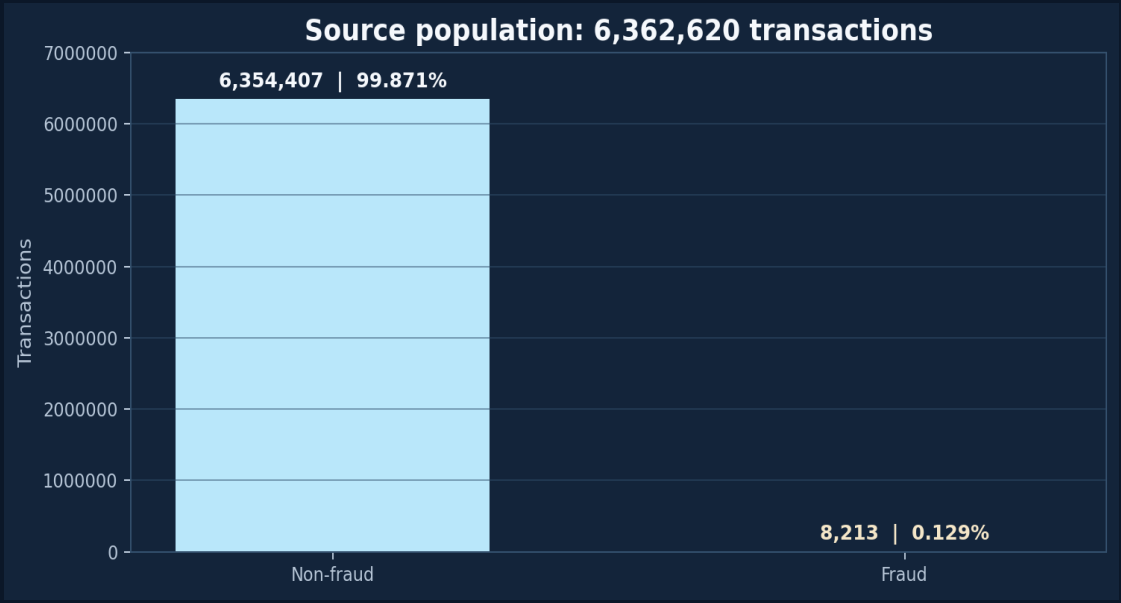
$L = -[y \log(p) + (1-y)\log(1-p)]$

Penalizes confident incorrect predictions.

SGD intuition: for each mini-batch, calculate the prediction error, compute the gradient, then update weights: $w \leftarrow w - \eta \cdot \nabla L(w)$. The learning rate η controls update size; regularization discourages excessively large weights.

Why class weights matter

If the majority class dominates the loss, a model can achieve high accuracy by mostly predicting normal.



WEIGHTED LOSS

Each class contributes according to its assigned weight; the rare fraud class receives greater influence during optimization.

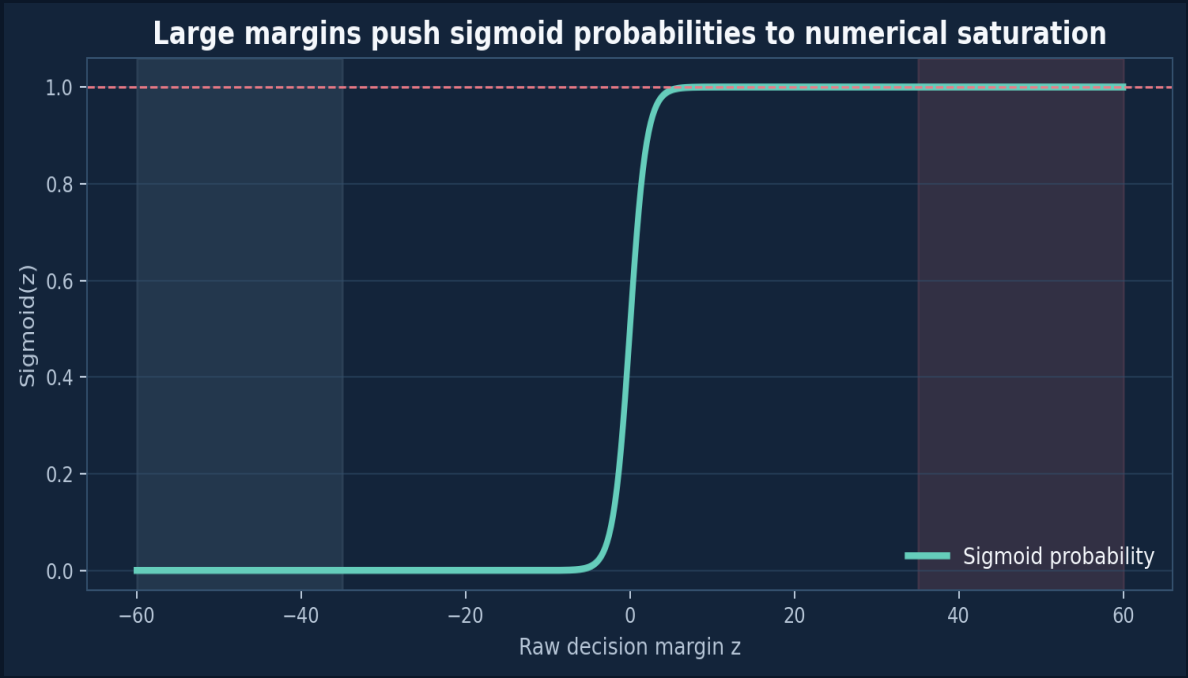
BALANCED-WEIGHT REFERENCE

A common reference formula is $w_i = N / (K \times n_i)$, where N is training rows, K is classes, and n_i is the count of class i.

Project fact: class weighting was applied during training. The exact configured numeric weights should be quoted from the saved training configuration, not guessed from the formula.

The diagnostic that changed the evaluation

Large raw margins pushed the sigmoid output close to 1.0, creating many tied probabilities.



WHAT HAPPENED

Initial validation probability output saturated near 1.0. Ranking information was compressed into ties, so probability-based Average Precision fell to 0.2046.

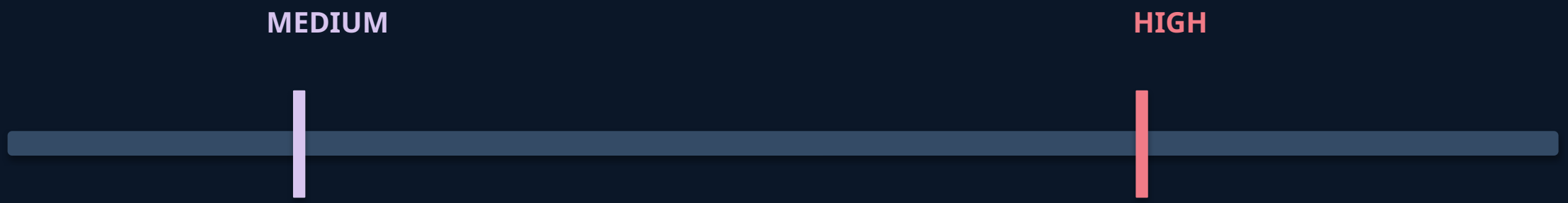
THE DIAGNOSTIC

Use `decision_function` raw margins to preserve the ordering of transactions. Raw-score Average Precision on validation was 0.995086.

A raw margin is not a probability. We use it as a ranking / threshold score and label it honestly in the dashboard.

One model score, two operational thresholds

Thresholds are selected on validation data and frozen before evaluating the final test set.



LOW

Score < 54.3042 → MONITOR

MEDIUM

54.3042 ≤ score < 213.8087 → MANUAL_REVIEW

HIGH

Score ≥ 213.8087 → ALERT_AND_SIMULATED_HOLD

Validation: threshold selection without touching test

The high-risk threshold was selected on the validation period.

Precision

1.0000

Recall

0.9941

F1

0.9970

Average Precision

0.9951

TP / FN

1,173 / 7

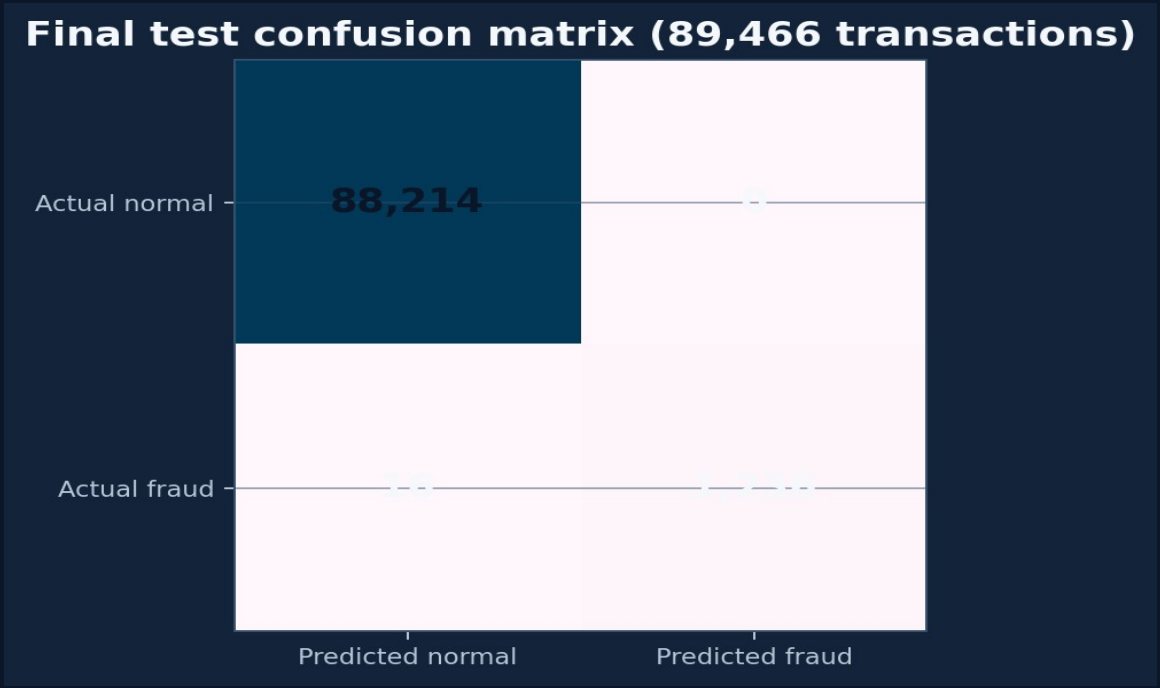
FP

0

Validation confusion matrix: TN 185,428 • FP 0 • FN 7 • TP 1,173. The validation set is for model / threshold choices, not the final unbiased estimate.

Final test: strong separation on the held-out future period

Fixed high threshold 213.80866687194845, chosen on validation, applied unchanged to test.



Accuracy
99.9821%

Precision
100.00%

Recall
98.7220%

F1
99.3569%

Average Precision
98.9589%

Precision lift
71.46×

Test counts: TN 88,214 • FP 0 • FN 16 • TP 1,236. Results describe this synthetic chronological holdout, not guaranteed performance on real bank traffic.

SQL Server: traceable evaluation store

Only validation + final-test rows were loaded to the dashboard database.

Transactions

TransactionID (PK)
SourceRowNumber (unique)
Step • TransactionType
Amount • NameOrig / NameDest
Balances • isFraud (evaluation label)

FraudPredictions

PredictionID (PK)
TransactionID (FK)
DatasetSplit
RawDecisionScore
PredictedFraud • RiskLevel
RecommendedAction
ModelName / ModelVersion

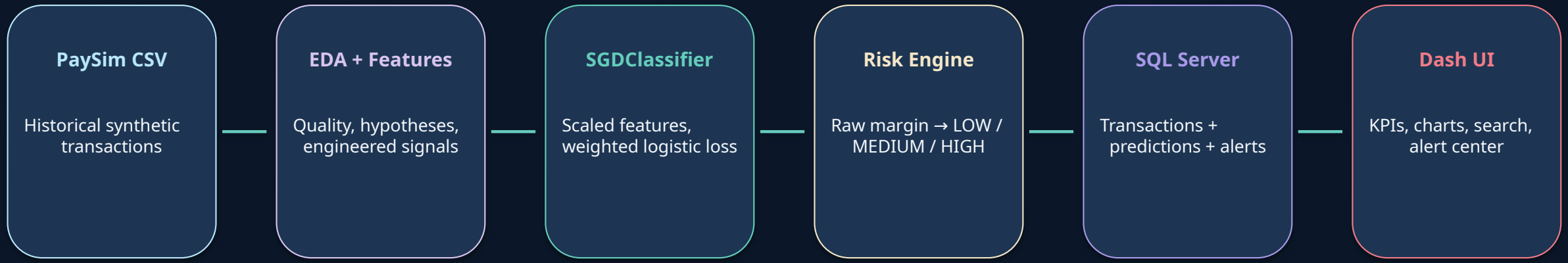
FraudAlerts

AlertID (PK)
PredictionID / TransactionID (FK)
AlertType • AlertStatus
AlertMessage • CreatedAtUtc
ResolvedAtUtc
SimulatedHold

Integrity audit: 280,613 transactions • 280,613 predictions • 5,255 alerts • 0 duplicate source row numbers • 0 orphan prediction / alert references.

From source data to analyst-facing decision support

A layered design keeps ML, persistence, risk policy and UI responsibilities separate.



The model produces a score. The risk engine applies versioned thresholds. The database preserves the decision. The dashboard makes the evidence inspectable. No real-time bank authorization is performed.

How the Dash dashboard works

The UI is reactive: component properties trigger callbacks, which query SQL and return new component properties.

1 • INPUTS

Dropdown filters
Refresh button
30-second interval
Search box
Selected table row

2 • CALLBACKS

Analytics callback
Transaction table callback
Transaction details callback
Alert table callback

3 • DATA ACCESS

Parameterized SQL
pyodbc connection
Pandas DataFrames
Error handling

4 • OUTPUTS

6 KPI cards
3 Plotly charts
Transaction table
Alert table
Details panel

Current integration test: 4 registered callbacks • 23 unique layout IDs • 0 missing IDs • 0 duplicate outputs.

What changes when I use each filter?

Current implementation behavior, based on the actual data-access signatures.

DATASET SPLIT

ALL / VALIDATION / TEST
Used by KPIs, charts, transaction explorer and alert center.

TRANSACTION TYPE

ALL / TRANSFER / CASH_OUT / CASH_IN / PAYMENT / DEBIT
Used by KPIs, risk distribution, temporal, explorer and alerts.

RISK LEVEL

ALL / LOW / MEDIUM / HIGH
Used by type analysis and transaction explorer. Not currently part of KPI SQL signature.

REFRESH

Manual button triggers refresh. `dcc.Interval` triggers refresh every 30 seconds. The controls refresh table outputs as well.

SEARCH + SELECT

Search matches SourceRowNumber, NameOrig or NameDest. Select one row to populate Transaction Details from the selected result record.

How to interpret the three charts

Every chart answers a different question; hover details provide additional context.

FRAUD BY TYPE

Bars compare actual fraud labels with model-predicted fraud. A line on a secondary axis shows total transaction volume.

Use it to compare categories without confusing count scales.

RISK DISTRIBUTION

A donut shows the number of transactions in LOW / MEDIUM / HIGH tiers. Hover shows actual fraud, predicted fraud and average amount.

It is a count distribution, not a probability chart.

FRAUD OVER TIME

Bars show transaction volume; lines show actual fraud, predicted fraud and a 7-step moving average.

The second axis prevents volume from flattening fraud counts.

Investigate one transaction from the table

The explorer fetches a bounded result set and shows a detailed evidence panel after row selection.

SEARCH

Search by SourceRowNumber or sender / receiver ID. SQL uses parameters for the search values.

Default query limit: 100 rows; data access enforces a maximum of 500.

TABLE

14 displayed fields: identity, step, type, amount, sender / receiver, split, raw score, predicted label, risk, action and alert data.

DETAILS

Shows transaction context, model decision, raw margin, risk tier, alert status and simulated hold.

`RawDecisionScore` is explicitly not labeled as probability.

From risk decision to an investigation queue

The Alert Center is backed by persisted alert records, not hard-coded demo cards.

ALERT RECORD

AlertID, category, status, message, creation / resolution times and transaction reference.

FILTERS

Dataset split, transaction type, alert status (OPEN / UNDER_REVIEW / RESOLVED) and alert category (HIGH_RISK / MANUAL_REVIEW).

AUTOMATED DISPLAY

Manual and 30-second refresh; results serialized for the DataTable. Current default result limit: 100 alerts.

What has been implemented and verified

Evidence-backed implementation status from project tests and dashboard execution.

DATA	6,362,620-row source profiled; engineered Parquet validated with 6,362,620 rows and 8,213 fraud labels.
MODEL	Validation threshold selected; untouched chronological test metrics computed.
DATABASE	280,613 evaluation rows loaded; 280,613 predictions; 5,255 alerts; referential integrity checked.
RISK + ALERTS	Risk engine smoke test covered LOW / MEDIUM / HIGH; JSONL alert routing verified.
DASH	3 charts tested with SQL data; 4 callbacks; 23 layout IDs; integration PASS; local server HTTP 200.

What the current evidence does not prove

A strong synthetic holdout result is not a guarantee of real-world financial performance.

Technical limits

- PaySim is synthetic; real bank behavior, customer mix and adversarial strategies may differ.
- Temporal fraud prevalence rises across the split, indicating distribution shift.
- Raw margins are not calibrated probabilities.
- Medium tier is based on a validation normal-score percentile, not an approved business cost policy.

Operational limits

- Balances after transaction make this post-transaction monitoring.
- No production bank integration or real transaction blocking.
- Alert status workflow is a prototype; human case management is future work.
- Cloud deployment requires remote database access or a cloud-compatible data layer.

Next phase: separate security labs

These are planned extensions, not completed components of the PaySim fraud model.

SQL INJECTION DEFENSIVE LAB

Goal: understand unsafe dynamic SQL, reproduce a harmless local test case, replace string concatenation with parameterized queries, add input validation, and retest.

Connection to this project: the existing data-access layer uses parameterized SQL values; the dedicated lab will deepen attack / defense testing.

MAN-IN-THE-MIDDLE DEFENSIVE LAB

Goal: build a controlled local simulation of message interception / alteration, use HMAC or signatures to detect tampering, reject invalid integrity checks, and log security events.

Scope: local educational simulation only; no real network interception.

Keep fraud detection, SQL injection testing and MITM integrity simulation as separate modules with separate datasets, tests and threat models.

From local dashboard to a shareable cloud link

GitHub stores source; a cloud web service runs Dash; a remote database provides the data.

1 • GITHUB

Create a clean repository. Commit source, SQL scripts, docs and safe sample outputs. Exclude `.venv`, credentials and large raw datasets.

2 • CLOUD HOST

Deploy a Python web service with Gunicorn / Docker and environment variables. The app listens on the provider-assigned PORT.

3 • REMOTE DATA

A local Windows SQL Server is not reachable from the cloud service by default. Use a secured hosted SQL endpoint or migrate to a cloud data store.

Render free web services sleep after 15 minutes without traffic and have ephemeral local storage. A cloud link is created only after repository connection, database configuration and successful deployment.

PAYSim FRAUD INTELLIGENCE PLATFORM

Detect • Explain • Prioritize • Monitor

A reproducible analytical prototype that turns synthetic transaction history into traceable model scores, risk tiers, SQL records and an interactive investigation dashboard.

DATA → MODEL | MODEL → RISK | RISK → SQL | SQL → DASHBOARD

Thank you • Questions

Sources behind the dataset and modeling choices

1. PaySim simulator paper

E. A. López-Rojas, A. Elmir, and S. Axelsson. “PaySim: A financial mobile money simulator for fraud detection.” 28th European Modeling and Simulation Symposium (EMSS), 2016.

https://www.msc-les.org/proceedings/emss/emss2016/emss2016_249.html

2. PaySim Kaggle dataset metadata

Synthetic Financial Datasets for Fraud Detection, ealaxi / PaySim1. The dataset metadata explains the synthetic data, fields and the warning about balance fields for fraud detection.

https://www.kaggle.com/datasets/ealaxi/paysim1/metadata

3. scikit-learn SGDClassifier

Official estimator documentation: logistic loss, class_weight, and incremental partial_fit learning.

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html

4. scikit-learn SGD user guide

Stochastic gradient descent, regularized linear models, and out-of-core / mini-batch learning.

https://scikit-learn.org/stable/modules/sgd.html

5. Render deployment references

Official Flask/Python deployment guide and free-service limitations; see the cloud deployment guide included with this submission.

https://render.com/docs/deploy-flask | https://render.com/docs/free

Use the dataset citation and license required by the data provider / course. Model metrics in this presentation are project experiment results, not claims from the references.